Guía de práctica: POO en Python

Sistema académico | Repaso para Anahí Robayo

Ejemplo unificado basado en las clases de tus prácticas. Incluye ocho clases, constructores, atributos, métodos, objetos, herencia, una clase abstracta y polimorfismo. El archivo Python adjunto contiene exactamente el código de esta guía.

1. Las ocho clases

Persona: clase abstracta con nombre e identificación.

Docente: hereda de Persona y define su actividad.

Estudiante: hereda de Persona y define otra actividad.

Asignatura: guarda código, nombre y horas semanales.

Curso: relaciona un docente y una lista de estudiantes.

Aula: guarda número, capacidad y tipo.

LaboratorioTI: hereda de Aula y añade equipos y tecnología.

Horario: relaciona un curso, una asignatura y un aula.

Cómo ejecutarlo

Abre practica_poo.py en tu editor y ejecútalo con Python 3. Solo utiliza la biblioteca estándar; no necesitas instalar paquetes. No requiere ingresar datos por teclado.

Este ejemplo integra los conceptos pertinentes al sistema académico. No añade NumPy ni herencia múltiple: no hacen falta para demostrar la abstracción y el polimorfismo de este modelo.

Qué debes poder explicar

Clase y objeto

La clase es el modelo. Docente es una clase; Docente("Carlos", "D01", "Manana") crea un objeto, al que se refiere la variable docente1.

Atributos y self

self representa la instancia que recibe la llamada. self.nombre guarda un dato propio de cada objeto. institucion y status se definen en la clase y se comparten mientras no se oculten con atributos de instancia.

Constructor y super()

__init__ inicializa los atributos. super().__init__(...) permite reutilizar la inicialización de la clase base.

Los tres métodos de tu práctica 2

Un método de instancia recibe self; un método con @classmethod recibe cls y trabaja con la clase; un método con @staticmethod no recibe automáticamente ninguno de los dos.

Abstracción

Persona hereda de ABC y declara realizar_actividad con @abstractmethod. No puedes instanciar Persona, ni una hija que mantenga métodos abstractos sin implementar. Una clase abstracta sí puede tener constructor, atributos y métodos concretos.

Herencia y sobrescritura

Docente y Estudiante son personas. LaboratorioTI es un aula y sobrescribe mostrar; super().mostrar() reutiliza la parte común.

Polimorfismo

En el bucle de personas se llama a realizar_actividad de la misma forma. El docente imparte clases y la estudiante realiza tareas. También el bucle de mostrar usa objetos de diferentes clases con una operación común.

Relación entre objetos

Un curso tiene un docente y estudiantes. Un horario tiene un curso, una asignatura y un aula. Estas relaciones se guardan como atributos; no requieren herencia.

En un archivo sencillo, define primero las clases base, después las hijas y al final crea los objetos. No redefinas varias veces una clase con el mismo nombre: la última definición reemplaza el nombre anterior.

Código completo | Parte 1

Continúa las partes en el mismo archivo, en este orden. Los saltos de página no forman parte del código.

```
"""Repaso de POO: sistema academico con ocho clases. Python 3."""
from abc import ABC, abstractmethod


# 1. CLASE ABSTRACTA: comparte datos y exige una actividad.
class Persona(ABC):
    institucion = "Universidad"  # Atributo de clase

    def __init__(self, nombre, identificacion):
        self.nombre = nombre  # Atributos de instancia
        self.identificacion = identificacion

    def mostrar(self):  # Metodo concreto heredado
        print(f"Nombre: {self.nombre} | ID: {self.identificacion}")

    @abstractmethod
    def realizar_actividad(self):
        pass


# 2. HERENCIA: Docente es una Persona.
class Docente(Persona):
    status = "Activo"

    def __init__(self, nombre, identificacion, disponibilidad):
        super().__init__(nombre, identificacion)
        self.disponibilidad = disponibilidad

    def realizar_actividad(self):  # Implementa el metodo abstracto
        print(f"El docente {self.nombre} imparte clases.")

    @classmethod
    def ver_estado(cls):  # Recibe la clase
        print(f"Estado del docente: {cls.status}")

    @staticmethod
    def mensaje():  # No necesita self ni cls
        print("Docente registrado")


# 3. Estudiante tambien hereda de Persona.
class Estudiante(Persona):
    estado = "Activo"

    def __init__(self, nombre, identificacion, matricula):
        super().__init__(nombre, identificacion)
        self.matricula = matricula

    def realizar_actividad(self):
```

Código completo | Parte 2

Continúa las partes en el mismo archivo, en este orden. Los saltos de página no forman parte del código.

```
        print(f"La estudiante {self.nombre} realiza tareas.")


# 4. ASIGNATURA
class Asignatura:
    tipo = "Academico"

    def __init__(self, codigo, nombre, horas_semanales):
        self.codigo = codigo
        self.nombre = nombre
        self.horas_semanales = horas_semanales

    def mostrar(self):
        print(f"Asignatura: {self.nombre} | Codigo: {self.codigo}")
        print(f"Horas semanales: {self.horas_semanales}")


# 5. CURSO: guarda un objeto Docente y una lista de estudiantes.
class Curso:
    nivel = "3 nivel"

    def __init__(self, nombre, paralelo, semestre, docente):
        self.nombre = nombre
        self.paralelo = paralelo
        self.semestre = semestre
        self.docente = docente
        self.estudiantes = []

    def agregar_estudiante(self, estudiante):
        self.estudiantes.append(estudiante)

    def mostrar(self):
        print(f"Curso: {self.nombre} | Paralelo: {self.paralelo}")
        print(f"Semestre: {self.semestre}")
        print(f"Docente: {self.docente.nombre}")
        for estudiante in self.estudiantes:
            print(f"Estudiante: {estudiante.nombre}")


# 6. AULA
class Aula:
    estado_general = "Disponible"

    def __init__(self, numero, capacidad, tipo):
        self.numero = numero
        self.capacidad = capacidad
        self.tipo = tipo

    def mostrar(self):
```

Código completo | Parte 3

Continúa las partes en el mismo archivo, en este orden. Los saltos de página no forman parte del código.

```
        print(f"Aula: {self.numero} | Capacidad: {self.capacidad}")
        print(f"Tipo: {self.tipo}")


# 7. HERENCIA SIMPLE Y SOBRESCRITURA DE mostrar().
class LaboratorioTI(Aula):
    def __init__(self, numero, capacidad, equipos, tecnologia):
        super().__init__(numero, capacidad, "Laboratorio TI")
        self.equipos = equipos
        self.tecnologia = tecnologia

    def mostrar(self):
        super().mostrar()
        print(f"Equipos: {self.equipos} | Tecnologia: {self.tecnologia}")


# 8. HORARIO: relaciona objetos de otras clases.
class Horario:
    modalidad = "Presencial"

    def __init__(self, dia, inicio, fin, curso, asignatura, aula):
        self.dia = dia
        self.inicio = inicio
        self.fin = fin
        self.curso = curso
        self.asignatura = asignatura
        self.aula = aula

    def mostrar(self):
        print(f"Horario: {self.dia}, {self.inicio} - {self.fin}")
        print(f"Curso: {self.curso.nombre}")
        print(f"Asignatura: {self.asignatura.nombre}")
        print(f"Aula asignada: {self.aula.numero}")


# PROGRAMA PRINCIPAL: crea objetos y utiliza sus metodos.
def main():
    docente1 = Docente("Carlos", "D01", "Manana")
    estudiante1 = Estudiante("Anahi", "E01", "M2026")
    asignatura1 = Asignatura("POO01", "Programacion orientada a objetos", 4)
    curso1 = Curso("Software", "A", 3, docente1)
    aula1 = Aula(101, 30, "Teorica")
    laboratorio1 = LaboratorioTI(202, 25, 25, "Python")
    horario1 = Horario("Viernes", "08:00", "10:00",
                       curso1, asignatura1, laboratorio1)

    curso1.agregar_estudiante(estudiante1)

    print("=== DATOS DEL PROYECTO ===")
```

Código completo | Parte 4

Continúa las partes en el mismo archivo, en este orden. Los saltos de página no forman parte del código.

```
    for elemento in [asignatura1, curso1, aula1, laboratorio1, horario1]:
        elemento.mostrar()
        print()

    print("=== POLIMORFISMO CON LA CLASE ABSTRACTA ===")
    personas = [docente1, estudiante1]
    for persona in personas:
        persona.mostrar()
        persona.realizar_actividad()

    print("\n=== METODOS DE CLASE Y ESTATICOS ===")
    Docente.ver_estado()
    Docente.mensaje()
    print(f"Institucion compartida: {Persona.institucion}")


if __name__ == "__main__":
    main()
```

Repaso antes de la práctica

Fragmento de la salida esperada

```
=== POLIMORFISMO CON LA CLASE ABSTRACTA ===
Nombre: Carlos | ID: D01
El docente Carlos imparte clases.
Nombre: Anahi | ID: E01
La estudiante Anahi realiza tareas.

=== METODOS DE CLASE Y ESTATICOS ===
Estado del docente: Activo
Docente registrado
Institucion compartida: Universidad
```

Ejercicio para hacer sin mirar

Añade una clase Coordinador que herede de Persona, guarde un área y defina realizar_actividad. Crea un objeto e inclúyelo en la lista personas. Comprueba que el mismo bucle ejecuta su actividad sin modificarlo. Esta ampliación tendrá nueve clases; el programa original tiene ocho.

Preguntas rápidas y respuestas

¿Puedo crear Persona("Ana", "P01")? No; falta una implementación concreta del método abstracto.

¿Qué ocurre si Docente no implementa realizar_actividad? Sigue siendo abstracta y al intentar crear un docente obtienes TypeError.

¿La clase abstracta es un tipo de polimorfismo? Son conceptos diferentes. La clase abstracta exige una operación; el polimorfismo permite usarla con comportamientos distintos. Python también admite polimorfismo sin una base común.

¿Esto incluye una interfaz independiente? No. Persona combina datos, código común y una operación requerida. En Python se puede representar una interfaz con una ABC que declare operaciones, pero no hace falta añadirla para este ejercicio.

¿Dónde van los métodos? Al mismo nivel de indentación que __init__, dentro de la clase. Si escribes def mostrar dentro de __init__, será una función local, no el método que buscas.

Orden para recordar: importar ABC y abstractmethod; definir la base abstracta; definir las hijas; definir las otras clases; crear objetos; llamar métodos; recorrer objetos para demostrar polimorfismo.